Step 2 — A Supervisor Routes Between Them — Solution

> Back to the step · Hint 1 · Hint 2

Story: this wires up a `Command`-based Supervisor (Project 4's pattern) inside a real LangGraph graph, routing between the two already-proven specialists from Step 1. The Supervisor's job is strictly to *route* — it never opens an MCP connection itself, or touches a specialist's internals. **If not:** the coordinator would end up reaching into each specialist's own MCP connection, and the clean "each specialist owns its own server" boundary this project depends on would be gone.

`src/state.py` (new)

```
# src/state.py
# The shared state. One writer per field: notes_agent writes notes_findings,
# web_agent writes web_findings + web_unavailable, writer writes report,
# supervisor writes routing_log.
from typing import TypedDict


class TeamState(TypedDict):
    task: str
    notes_findings: str
    web_findings: str
    web_unavailable: bool
    report: str
    routing_log: list


def start_state(task):
    return {
        "task": task,
        "notes_findings": "",
        "web_findings": "",
        "web_unavailable": False,
        "report": "",
        "routing_log": [],
    }
```

`src/agents/supervisor.py` (new)

Story: `decide_next` is a plain function of the state — no model call — that names exactly who runs next, based on what's still missing and what the task actually asked for. `supervisor()` wraps it as a node: logs the decision, then routes with `Command`. **If not:** routing logic scattered directly through the graph's edges would be impossible to unit-test without a real model call every time.

```
# src/agents/supervisor.py
# decide_next: route to the next specialist that still has work, else the
# writer. supervisor: log the decision and route with Command. This never
# touches an MCP connection -- it only picks a node.
from langgraph.types import Command

from src.utils.logger import get_logger


def needs_notes(task):
    lowered = task.lower()
    return "note" in lowered or "check" in lowered or "both" in lowered


def needs_web(task):
    lowered = task.lower()
    return "fetch" in lowered or "status page" in lowered or "web" in lowered


def decide_next(state):
    task = state["task"]
    if needs_notes(task) and state["notes_findings"] == "":
        return "notes"
    if needs_web(task) and state["web_findings"] == "" \
            and not state["web_unavailable"]:
        return "web"
    return "writer"


def supervisor(state):
    logger = get_logger(__name__)
    target = decide_next(state)
    log = state["routing_log"] + [target]
    logger.info(f"supervisor -> {target}")
    return Command(goto=target, update={"routing_log": log})
```

`src/graph.py` — the async graph

Story: the only fixed edge in this graph is `START → supervisor`. The specialist nodes are themselves declared **async** (since they open real MCP connections), and the whole graph is run with `.ainvoke()` to match. Each specialist node calls its agent and returns `Command(goto="supervisor", ...)`, so control always flows back to the Supervisor to decide what happens next. **If not:** a sync node wrapping the async specialist in `asyncio.run()` would fight LangGraph's own event loop and hang.

```
# src/graph.py  (Step 2 version — no writer yet; the writer target ends the
# run until Step 3 adds the node)
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command

from src.agents.notes_agent import run_notes_agent
from src.agents.supervisor import supervisor
from src.agents.web_agent import run_web_agent
from src.state import TeamState


def build_graph(client, connect_timeout=10.0):
    async def notes_node(state):
        findings, _ = await run_notes_agent(
            client, state["task"], connect_timeout
        )
        return Command(goto="supervisor",
                       update={"notes_findings": findings})

    async def web_node(state):
        findings, unavailable = await run_web_agent(
            client, state["task"], connect_timeout
        )
        return Command(goto="supervisor", update={
            "web_findings": findings, "web_unavailable": unavailable,
        })

    def writer_node(state):
        # placeholder until Step 3: just concatenate the raw findings
        return Command(goto=END, update={
            "report": state["notes_findings"] + "\n" + state["web_findings"],
        })

    graph = StateGraph(TeamState)
    graph.add_node("supervisor", supervisor)
    graph.add_node("notes", notes_node)
    graph.add_node("web", web_node)
    graph.add_node("writer", writer_node)
```

```
    graph.add_edge(START, "supervisor")
    return graph.compile()
```

In `main.py`, build the graph and run it with `await build_graph(client).ainvoke(start_state(task))`.

The test

```
# tests/unit/test_supervisor.py
# The routing brain, on hand-made states — no model, no servers.
# Run from the repo root: python -m tests.unit.test_supervisor
from src.agents.supervisor import decide_next
from src.state import start_state

BOTH = "Check my notes for Atlas, fetch its status page, and give a report."


def test_fresh_routes_to_notes():
    target = decide_next(start_state(BOTH))
    result = "WRONG"
    if target == "notes":
        result = "OK"
    print(f"case 1 (fresh -> notes)  -> {result}")


def test_notes_done_routes_to_web():
    state = start_state(BOTH)
    state["notes_findings"] = "found"
    target = decide_next(state)
    result = "WRONG"
    if target == "web":
        result = "OK"
    print(f"case 2 (notes done -> web) -> {result}")


def test_both_done_routes_to_writer():
    state = start_state(BOTH)
    state["notes_findings"] = "found"
    state["web_findings"] = "fetched"
    target = decide_next(state)
    result = "WRONG"
    if target == "writer":
        result = "OK"
    print(f"case 3 (both done -> writer) -> {result}")
```

```
def test_web_unavailable_skips_web():
    state = start_state(BOTH)
    state["notes_findings"] = "found"
    state["web_unavailable"] = True
    target = decide_next(state)
    result = "WRONG"
    if target == "writer":
        result = "OK"
    print(f"case 4 (web down -> writer) -> {result}")


if __name__ == "__main__":
    test_fresh_routes_to_notes()
    test_notes_done_routes_to_web()
    test_both_done_routes_to_writer()
    test_web_unavailable_skips_web()
```

Expected output:

```
case 1 (fresh -> notes)  -> OK
case 2 (notes done -> web) -> OK
case 3 (both done -> writer) -> OK
case 4 (web down -> writer) -> OK
```

Case 4 is the one that matters for Step 4: once the Web Agent has marked its server unavailable, the Supervisor routes straight to the Writer instead of looping on a server that isn't coming back.